Claude Code 深度拆解：一个顶级AI编程工具的核心架构

茂道 大淘宝技术 2025年9月22日 01:47 浙江 32人

文章深度拆解了Anthropic开发的终端AI编程工具Claude Code，包括其以交互层、执行层、核心引擎为核心的系统架构，提交命令到渲染结果的执行流程，以及交互层的输入处理与渲染、核心引擎的消息与查询管理、工具系统、上下文管理（含LRU缓存等策略）和安全机制，还分享了Binary Feedback测试机制、MCP工具分层管理等技术启发；同时介绍了心流团队基于Gemini CLI改造、融合Claude Code特性的iFlow CLI 2.0，包括其安装方式、功能特点（如多运行模式、SubAgent功能等）、开放市场资源，以及CLI工具在项目开发、网站制作、DeepResearch等场景的应用，还给出了西欧亲子自驾游规划案例。

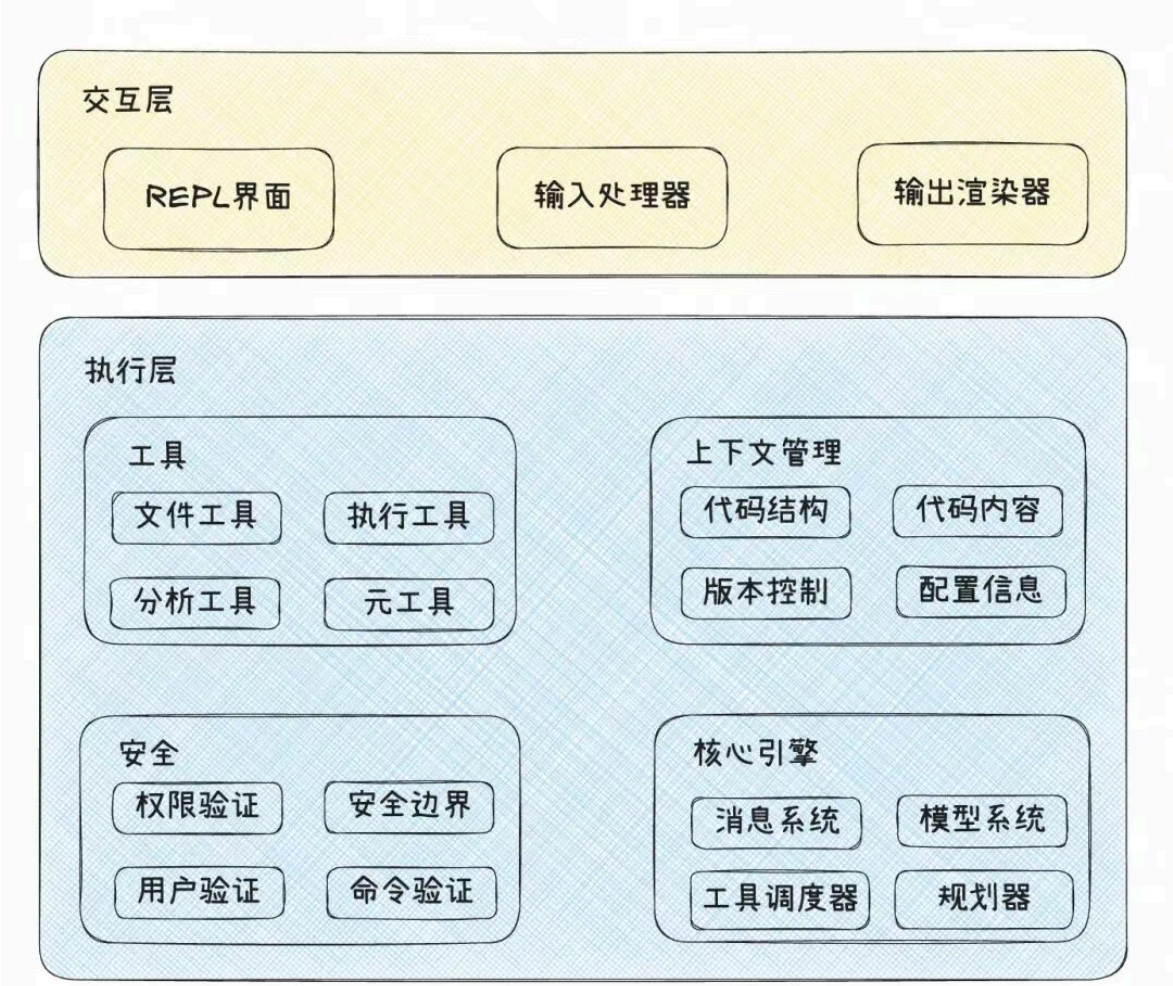
01 Claude Code介绍

ClaudeCode是由Anthropic开发的全新终端AI编程工具，旨在通过自然语言指令帮助开发者高效率地完成代码编写、调试和项目管理任务。它直接集成在开发者的工作环境（如终端）中，无需依赖额外服务器或复杂配置即可运行。

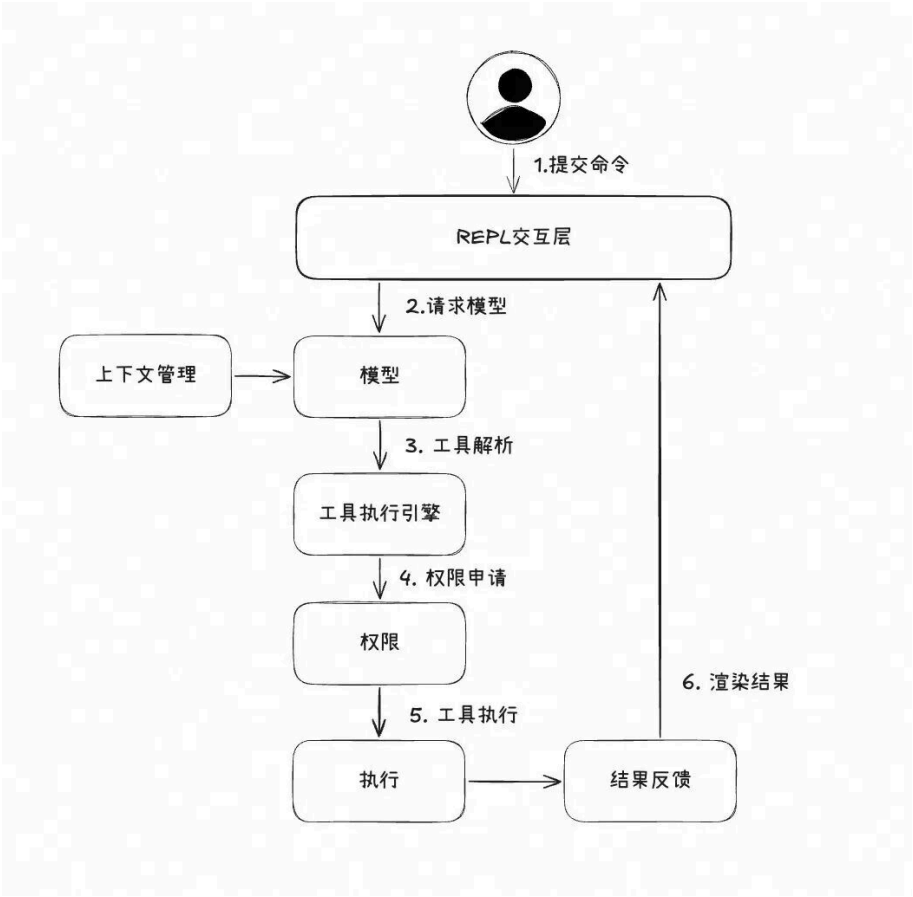
在实际的使用过程中，Claude Code是一个比较通用的智能体，他输出的代码也比cursor简练很多，更像是一个熟悉整个项目的高级程序员，研究和学习这个框架对于开发自己的Agent至关重要，本篇文章会详细介绍Claude Code的设计模式和核心代码。

02 详细介绍

2.1系统架构


交互层
REPL界面
输入处理器
输出渲染器
执行层
工具
文件工具
执行工具
分析工具
元工具
上下文管理
代码结构
代码内容
版本控制
配置信息
安全
权限验证
安全边界
用户验证
命令验证
核心引擎
消息系统
模型系统
工具调度器
规划器

2.2执行流程


1.提交命令
REPL交互层
2.请求模型
上下文管理
模型
3. 工具解析
工具执行引擎
4. 权限申请
权限
5. 工具执行
执行
结果反馈
6. 渲染结果

2.3交互层

交互层是用户与 Claude Code 的接触点，通常包括：

- REPL 界面：提供命令行交互体验；
- 输入处理器：解析用户指令，支持多种输入模式（自然语言、命令、代码等）；
- 输出渲染器：格式化并展示 AI 响应和工具执行结果；

在 ClaudeX 中，这一层主要由REPL.tsx和PromptInput.tsx等组件实现，它们负责接收用户输入并展示结果。

2.3.1输入逻辑处理

输入逻辑处理

```
function processUserInput(input: string, mode: InputMode): UserMessage
  if (input.startsWith('/')) {
    return handleSlashCommand(input);
  } elseif (input.startsWith('!')) {
    return handleBashCommand(input.slice(1));
  } else {
    return createUserMessage(input);
  }
}
```

2.3.2渲染

- 阶段
  - 工具调用阶段（Assistant侧）：显示工具名称、参数和执行状态；
  - 工具结果阶段（User侧）：显示执行结果；

- 组件
  - AssistantToolUseMessage:渲染助手调用工具时的消息；
    - 显示工具名称（通过 tool.userFacingName()）；
    - 显示参数（通过 tool.renderToolUseMessage()）；
    - 使用 ToolUseLoader 显示执行状态动画；
  - UserToolSuccessMessage:
    - 渲染工具执行成功后的结果；
    - 调用 tool.renderToolResultMessage() 渲染具体内容；

- 工具接口定义
  - userFacingName() 显示给用户的工具名称；
  - renderToolUseMessage(input, options) 渲染工具参数；
  - renderToolResultMessage(output, options)渲染工具结果；
  - renderToolUseRejectedMessage() 渲染拒绝消息；

- UI渲染特性
  - 使用 Ink 框架的 Box 和 Text 组件构建终端UI；
  - 支持主题系统（通过 getTheme()）；
  - 响应式布局（flexDirection, justifyContent）；
  - 详细/简洁模式切换（verbose 参数）；
  - 执行成本和时间显示（Cost 组件）；

2.4核心引擎

核心引擎是 Claude Code 的"大脑"，负责协调各个组件的工作：

- 消息系统：管理用户输入、AI 响应和工具结果的消息流；
- 查询引擎：与 AI 模型交互，发送请求并处理响应；
- 工具调度器：协调工具的调用和结果处理；

在 ClaudeX 中，query.ts是核心引擎的关键组件，它实现了与 AI 模型交互的逻辑：

query.ts

```
// 简化的查询逻辑
async function* query(
  messages: Message[],
  systemPrompt: string,
  context: Context,
  canUseTool: CanUseToolFn,
  toolUseContext: ToolUseContext,
): AsyncGenerator<Message> {
  ## 1. 初始化大模型prompt
  const fullSystemPrompt = formatSystemPromptWithContext(systemPrompt,

  ## 2. 获取大模型返回
  const result = await queryWithBinaryFeedback(
    toolUseContext,
    getAssistantResponse,
    getBinaryFeedbackResponse,
  )

```

```
  ## 3. 处理模型返回的工具请求
  const toolUseMessages = assistantMessage.message.content.filter(
    _ => _.type === 'tool_use',
  )

  ## 4. 并行/串行执行工具
  if (
    toolUseMessages.every(msg =>
      toolUseContext.options.tools.find(t => t.name === msg.name)?.isR
    )
  ){
    for await (const message of runToolsConcurrently
  } else{
    for await (const message of runToolsSerially
  }

  ## 5. 处理后续交互
  yield* await query()
}
```

2.5工具

工具系统是 Claude Code 的"手脚"，使其能够与外部环境交互：

- 文件工具：读取、写入、搜索文件；
- 执行工具：运行 shell 命令、执行代码；
- 分析工具：代码分析、依赖检查等；
- 元工具：复合工具，可以执行更复杂的任务；

每个工具都遵循统一的接口，包括名称、描述、参数模式和执行逻辑：

```
interface Tool {
  name: string;
  description: string;
  inputSchema: z.ZodType;
  execute(params: any): Promise<ToolResult>;
}
```

工具也是整个Claude Code的核心资产，他这个cli效果这么好除了是因为强大的模型，也是因为有很强大的工具，例如有一个特别强大的bash tool的工具，可以调用shell里面的所有命令，也包含agent tool，可以发挥更强大的能力。我们会在后续的文章中对工具进行单独的分析。

2.6上下文管理

上下文管理是 Claude Code 的"记忆"，负责收集和组织代码相关信息：

- 项目结构：目录和文件结构
- 代码内容：关键文件的内容
- 版本控制：Git 历史和状态
- 配置信息：项目配置和依赖

上下文管理的挑战在于如何在有限的上下文窗口内提供最相关的信息：

- LRU缓存机制：对文件编码、行尾类型等信息实现缓存，减少重复操作。

文件缓存

```
const fileEncodingCache = new LRUCache<string, BufferEncoding>({
  fetchMethod: path => detectFileEncodingDirect(path),
  ttl: 5 * 60 * 1000,
  ttlAutopurge: false,
  max: 1000,
})

const lineEndingCache = new LRUCache<string, LineEndingType>({
  fetchMethod: path => detectLineEndingsDirect(path),
  ttl: 5 * 60 * 1000,
  ttlAutopurge: false,
  max: 1000,
})
```

- 按需加载策略：不会一次性加载整个代码库，而是根据查询需要智能加载相关文件 。

GlobTool.tsx

```
async *call({ pattern, path }, { abortController }){
    const start = Date.now()
    const { files, truncated } = await glob(
      pattern,
      path ?? getCwd(),
      { limit: 100, offset: 0 },
      abortController.signal,
    )
    const output: Output = {
      filenames: files,
      durationMs: Date.now() - start,
```

```
        numFiles: files.length,
        truncated,
      }
      yield {
        type: 'result',
        resultForAssistant: this.renderResultForAssistant(output),
        data: output,
      }
    },
```

- 结果截断处理：对大量搜索结果实现智能截断，避免上下文溢出，同时提供清晰的截断提示 。

lsTool.tsx

```
const MAX_LINES = 4
const MAX_FILES = 1000
const TRUNCATED_MESSAGE = `There are more than ${MAX_FILES} files in th
```

- 拼装上下文

上下文拼装

```
async function getContext(): Context {
  return {
    directoryStructure: await getDirectoryStructure(),
    gitStatus: await getGitStatus(),
    codeStyle: await getCodeStyle(),
    // 其他上下文...
  };
}
```

2.7安全

安全与权限是 Claude Code 的"护栏"，确保工具使用的安全性：

- 权限验证：工具执行前的权限检查；
- 用户确认：关键操作的用户确认机制。使用最小权限原则，只向用户索要完成任务的最小权限；
- 安全边界：文件操作和命令执行的限制；

permission.ts

```
exportconst hasPermissionsToUseTool
```

```
  // 跳过
  if (context.options.dangerouslySkipPermissions) {
    return { result: true }
  }

  if (context.abortController.signal.aborted) {
    thrownew AbortError()
  }

  // 允许每个工具单独配置，由工具声明安全性，因为cc的工具都是自己实现的，他可以自己
  try {
    if (!tool.needsPermissions(input as never)) {
      return { result: true }
    }
  } catch (e) {
    logError(`Error checking permissions: ${e}`)
    return { result: false, message: 'Error checking permissions' }
  }

}
```



3.1用于程序员测试prompt的Binary Feedback机制

query.ts

```
async function queryWithBinaryFeedback(
  toolUseContext: ToolUseContext,
  getAssistantResponse: () => Promise<AssistantMessage>,
  getBinaryFeedbackResponse?: (
    m1: AssistantMessage,
    m2: AssistantMessage,
  ) => Promise<BinaryFeedbackResult>,
): Promise<BinaryFeedbackResult> {
  if (
    process.env.USER_TYPE !== 'ant' ||
    !getBinaryFeedbackResponse ||
    !(await shouldUseBinaryFeedback())
  ):
  const [m1, m2] = await Promise.all([
```

```
    getAssistantResponse(),
    getAssistantResponse(),
  ])
}
```

使用完全一模一样的请求两次，可能是为了观察同样的输入模型会不会给出两个输出，如果ai返回了两个答案，就说明模型对于这次请求是犹豫的，考虑不清楚的，需要让用户去做选择。

这里对于输出的检测不是检测文本，而是去检测生成的tool use是不是相同的。结构化数据输出的稳定性要远远大于文本输出的稳定性。如果没有tool use才会去比较文本。

同时看到这个代码只在程序员自己测试的时候才会生效，说明这样的测试逻辑对于开发自己的agent还是比较有用的。出现这种情况就说明你需要去提高自己的prompt质量了。

3.2MCP工具

整个Claude Code自身只维护了一个tengu_mcp_server，同时支持用户添加MCP Server，Claude Code通过三级分层的结构，来管理用户的MCP 工具：

- global：全局配置的通用的MCP
- MCPrc：配置文件，可以共享
- project：项目级别，就是当前的代码库可以单独配置

下层结构的MCP配置的可以覆盖上层的MCP配置。

添加MCP Server

```
export function addMcpServer(
  name: McpName,
  server: McpServerConfig,
  scope: ConfigScope = 'project',
): void {
  if (scope === 'mcprc') {
    if (process.env.NODE_ENV === 'test') {
      addMcprcServerForTesting(name, server)
    } else {
      const mcprcPath = join(getCwd(), '.mcprc')
      let mcprcConfig: Record<string, McpServerConfig> = {}

      // Read existing config if present
      if (existsSync(mcprcPath)) {
        try {
```

```
          const mcprcContent = readFileSync(mcprcPath, 'utf-8')
          const existingConfig = safeParseJSON(mcprcContent)
          if (existingConfig && typeof existingConfig === 'object') {
            mcprcConfig = existingConfig as Record<string, McpServerCo
          }
        } catch {
          // If we can't read/parse, start with empty config
        }
      }

      // Add the server
      mcprcConfig[name] = server

      // Write back to .mcprc
      try {
        writeFileSync(mcprcPath, JSON.stringify(mcprcConfig, null, 2),
      } catch (error) {
        thrownew Error(`Failed to write to .mcprc: ${error}`)
      }
    }
  } elseif (scope === 'global') {
    const config = getGlobalConfig()
    if (!config.mcpServers) {
      config.mcpServers = {}
    }
    config.mcpServers[name] = server
    saveGlobalConfig(config)
  } else {
    const config = getCurrentProjectConfig()
    if (!config.mcpServers) {
      config.mcpServers = {}
    }
    config.mcpServers[name] = server
    saveCurrentProjectConfig(config)
  }
}
```

如何获取到多个MCP Server的工具，请求所有的MCP Server，获取到所有的tools，将tools聚合，发送给大模型。

getMCPTools

```
exportconst getMCPTools = memoize(async (): Promise<Tool[]> => {
  const toolsList = await requestAll<
    ListToolsResult,
```

```
    typeof ListToolsResultSchema
  >(
    {
      method: 'tools/list',
    },
    ListToolsResultSchema,
    'tools',
  )
```

3.3通过AI来检测安全

利用AI做安全辅助。例如判断命令是否有被注入的可能性。
判断命令是否有被注入

```
exportconst getCommandSubcommandPrefix = memoize(
  async (
    command: string,
    abortSignal: AbortSignal,
  ): Promise<CommandSubcommandPrefixResult | null> => {
    const subcommands = splitCommand(command)

    ## 获取命令前缀来观察是否在安全执行列表
    const [fullCommandPrefix, ...subcommandPrefixesResults] = await Pr
      [
        getCommandPrefix(command, abortSignal),
        ...subcommands.map(async subcommand => ({
          subcommand,
          prefix: await getCommandPrefix(subcommand, abortSignal),
        })),
      ],
    )

  },
  command => command, // memoize by command only
)
```

3.4上下文压缩处理

核心功能：清空对话历史但保留上下文摘要，解决了长对话导致的上下文窗口问题。

技术亮点：

1. 智能摘要生成：使用Sonnet模型生成对话摘要，保留关键信息供后续使用；
2. Fork机制应用：利用setForkConvoWithMessagesOnTheNextRender创建新的对话分支，摘要作为新对话的起点；
3. Token使用优化：将摘要的token使用量设为0，避免触发上下文窗口警告；
4. 缓存清理：清理getContext和getCodeStyle缓存，确保新对话环境干净；

核心代码流程：

1. 获取当前对话历史
2. 构造摘要请求并调用Sonnet模型
3. 提取并验证摘要内容
4. 清屏、清空消息历史
5. 创建包含摘要的新对话分支

3.5简单任务交给小模型

Claude内部会有好几个模型实例，如果是只判断对错或者是一些简单的任务，会交给haiku模型。

queryHaiku

```
## 创建标题任务
async function generateTitle(description: string): Promise<string> {
  const response = await queryHaiku({
    systemPrompt: [
      'Generate a concise issue title (max 80 chars) that captures the
    ],
    userPrompt: description,
  })
  const title =
    response.message.content[0]?.type === 'text'
      ? response.message.content[0].text
      : 'Bug Report'
  if (title.startsWith(API_ERROR_MESSAGE_PREFIX)) {
    return `Bug Report: ${description.slice(0, 60)}${description.lengt
  }
  return title
}

## 确认前缀的任务
const getCommandPrefix = memoize(
  async (
    command: string,
```

```
    abortSignal: AbortSignal,
  ): Promise<CommandPrefixResult | null> => {
    const response = await queryHaiku({
      systemPrompt: [
        `Your task is to process Bash commands that an AI coding agent

This policy spec defines how to determine the prefix of a Bash command
      ],

Examples:
- cat foo.txt => cat
- cd src => cd
- cd path/to/files/ => cd
- find ./src -type f -name "*.ts" => find
- gg cat foo.py => gg cat
- gg cp foo.py bar.py => gg cp
- git commit -m "foo" => git commit
- git diff HEAD~1 => git diff
- git diff --staged => git diff
- git diff $(pwd) => command_injection_detected
- git status => git status
```

3.6高效的文件系统策略

- 分层的代码项目拉取，先获取高层次项目结构，再根据需要深入特定目录，避免一次性加载过多内容；
- Ripgrep集成：利用Rust编写的高性能搜索工具ripgrep，实现毫秒级的代码库搜索；
- 内置二进制支持：包含预编译的ripgrep二进制文件，确保跨平台一致性和性能；
- 智能结果排序：搜索结果按修改时间排序，优先展示最近更改的文件，提高相关性；
- 并发搜索能力：支持同时发起多个搜索请求，大幅提高大型代码库的分析效率；
- LRU缓存机制：对文件编码、行尾类型等信息实现缓存，减少重复操作；

3.7精妙的工具

Claude Code中内置的15个工具的实现和提示词都值得反复学习，就是因为这些极致的工具才使得任务执行的高效。



```
/**
 * The rules of thinking are lengthy and fortuitous. They require plent
 * of most long duration and deep meditation for a wizard to wrap one's
 *
 * The rules follow:
 * 1. A message that contains a thinking or redacted_thinking block mus
 * 2. A thinking block may not be the last message in a block
 * 3. Thinking blocks must be preserved for the duration of an assistar
 *
 * Heed these rules well, young wizard. For they are the rules of thinl
 * the rules of thinking are the rules of the universe. If ye does not
 * rules, ye will be punished with an entire day of debugging and hair
 */
```

05 iFlow CLI

随着Claude Code的爆火出圈，我们发现这个产品是真实地提高程序员的开发效率。我们也持续追踪了两个月Claude Code的升级进展，帮助了很多同学在国内环境使用Claude Code。随着Cursor、Claude Code对国内ip加大力度管控，使用这类前沿工具变得十分困难，但最近一个月也迎来了很多惊喜：Kimi K2、Qwen3-Coder、GLM-4.5 等国产大模型相继涌现，Gemini CLI 也宣布开源。国产编程工具正不断突破创新，致力于为国内开发者打造更智能、更高效的极致开发体验。

心流团队一直在探索AGI的路上，我们也一直在找寻最通用AI助手的技术架构和产品形态，CLI + MCP的出现让我们觉得这可能就是我们实现AGI的路径。所以我们向大家隆重介绍我们最新研发的CLI工具，iFlow CLI 2.0。



它基于Gemini CLI改造，我们花了一些时间打磨它，融合了一些Claude Code的特性。

Feature	iFlow Cli	Claude Code	Gemini Cli
Todo Planning	✅	✅	❌
SubAgent	✅	✅	❌
Custom Commands	✅	✅	✅
Plan Mode	✅	✅	❌
Task Tools	✅	✅	❌
VS Code Plugin	✅	✅	✅
JetBrains Plugin	✅	✅	❌
Conversation Recovery	✅	✅	❌
Built-in Open Market	✅	❌	❌
Memory Auto-compression	✅	✅	✅
Multimodal Capability	✅	⚠️ (Limited in China)	⚠️ (Limited in China)
Search	✅	❌	⚠️ (Requires VPN)
Free	✅	❌	⚠️ (Limited Usage)

- 支持4种运行模式。yolo（模型最大权限，可以执行任何操作），accepting edits(模型只有修改文件权限)，plan mode(先规划后执行)，default(模型无任何权限)；
- 升级subAgent功能。让CLI从通用助手向专家团队进化，为你提供更加专业、更加准确的建议，使用/agent可以看到更多预先配置好的agent；
- 升级task工具。有效压缩上下文长度，让CLI能够更加深入完成你的任务，上下文达到70%的时候会自动压缩；
- 接入心流开放市场。快速安装好用的MCP工具、Subagent和自定义指令，可以通过/mcp了解更多；
- 免费使用多模态模型，也可以在CLI中粘贴图片了（Ctrl+V粘贴图片）；
- 支持历史对话保存和回滚(iflow --resume和/chat命令）；
- 支持更多更好用终端命令（iflow -h查看更多命令）；

- 支持VSCode、Jetbrains插件(注意IDE对应版本号)；
- 自动升级，再也不用担心最新功能体验不到了；

5.1 一键安装

mac/linux/ubuntu用户

```
# 一键安装脚本，会安装全部所需依赖
bash -c "$(curl -fsSL https://gitee.com/iflow-ai/iflow-cli/raw/main/ins

# 已有Node.js 22+
npm i -g @iflow-ai/iflow-cli@latest
```

已经安装过的同学可以再次执行这个脚本，实现更新，后续cli会自动更新。

Windows用户

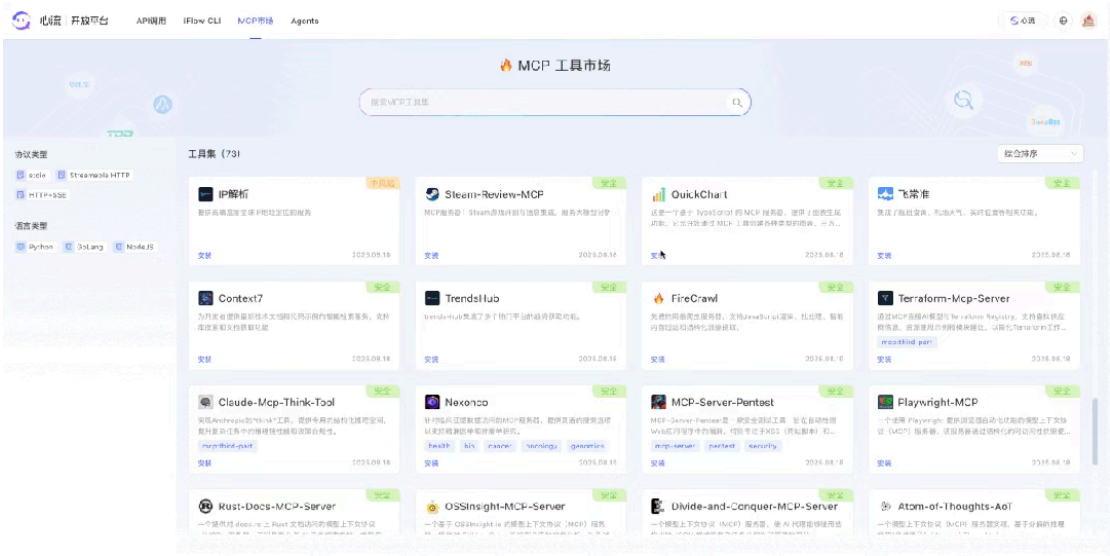
1. 访问 https://nodejs.org/zh-cn/download 下载最新的 Node.js 安装程序；
2. 运行安装程序来安装 Node.js；
3. 重启终端：CMD 或 PowerShell；
4. 运行 `npm install -g @iflow-ai/iflow-cli` 来安装 iFlow CLI；
5. 运行 `iflow` 来启动 iFlow CLI；

5.2 开放市场安装SubAgent和MCP工具

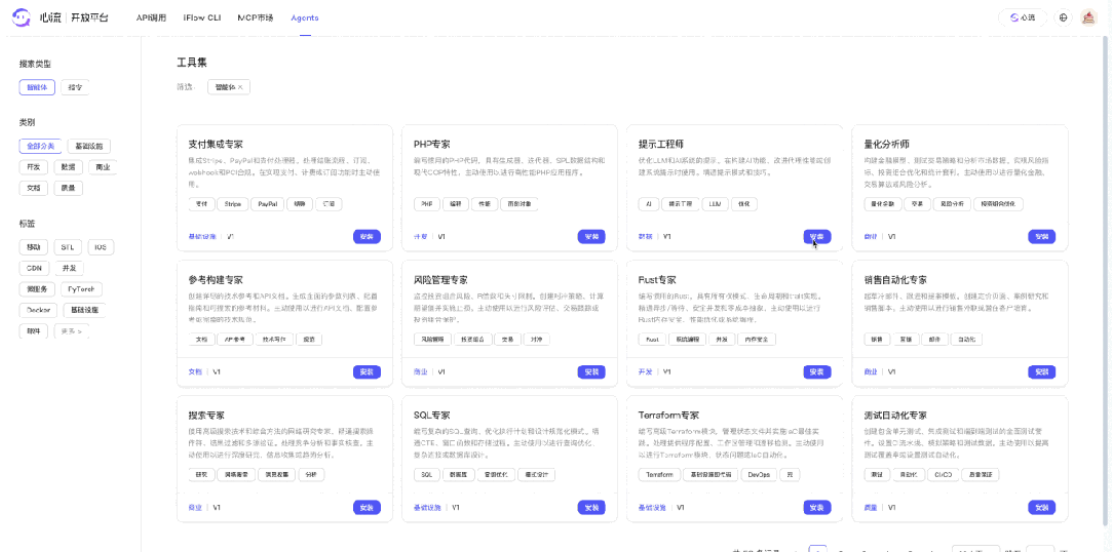
缺少了MCP和Sub agent的CLI是不完整的，加入MCP和Sub agent的之后会让CLI变成领域专家！对于很多小白用户来说，操作实在太难了！

心流开放市场帮你解决 👉 立即体验：https://platform.iflow.cn/cli

MCP开放市场，一键安装，解锁高效工具。



agent开放市场，一键安装Subagent，引入更强大的专家agent。



复制安装命令！在终端中执行！一键安装体验MCP和Subagent！



6.1 熟悉项目、写代码、debug

这是CLI的强项，进入到你的项目中，打开iFlow CLI，什么功能不熟悉，问他；什么业务需要实现，描述清楚让CLI帮你实现；遇到报错了，把报错信息贴给CLI让它帮你解决。

6.2 做网站

使用本地的私有数据，绘制你自己的报告网站。



6.3 DeepReseach

```
你会根据用户的问题/需求，根据任务执行的顺序收集信息，最终以HTML的方式产出一份报告。
  ## 任务执行顺序
  - 你会生成非常详细的信息搜集计划。
  - 然后根据信息搜集计划生成一个html template作为index.html，他包含了一些顶部菜
  - 根据信息搜集计划设计index.html的顶部菜单
  - 接着，你会按照信息搜集计划挨个进行信息收集，每收集一份信息，即生成对应的sectior
  - 插入过程中，你需要关注整体页面的样式，修复一些常见的显示错误，保障整体的风格一致
  - 不断进行信息收集、section代码生成、html template内容填充，直到所有信息收集完
  - MUST：确保按照计划完成所有的action
  ## 对于最终的html产出
  - 设计趋势和最佳实践有深入理解，尤其擅长创造具有极高审美价值的用户界面。你的设计作
  - 可能包括HTML、CSS、和JavaScript文件。不断根据任务，优化这个网页项目，使网页内
  # 设计要求
  **设计目标：**
  *   **视觉吸引力：** 创造一个在视觉上令人印象深刻的网页，能够立即吸引用户的注意力
  *   **可读性：** 确保内容清晰易读，无论在桌面端还是移动端，都能提供舒适的阅读体验
  *   **信息传达：** 以一种既美观又高效的方式呈现信息，突出关键内容，引导用户理解核
  *   **情感共鸣:** 通过设计激发与内容主题相关的情感（例如，对于励志内容，激发积极
  **设计指导（请灵活运用，而非严格遵循）：**
  *   **整体风格：** 可以考虑杂志风格、出版物风格，或者其他你认为合适的现代 Web 设
  *   **Hero 模块（可选，但强烈建议）：** 如果你认为合适，可以设计一个引人注目的
  *   **排版：**
      *   精心选择字体组合（衬线和无衬线），以提升中文阅读体验。
      *   利用不同的字号、字重、颜色和样式，创建清晰的视觉层次结构。
      *   可以考虑使用一些精致的排版细节（如首字下沉、悬挂标点）来提升整体质感。
      *   Font-Awesome中有很多图标，选合适的点缀增加趣味性。
  *   **配色方案：**
```

```
        *   选择一套既和谐又具有视觉冲击力的配色方案。
        *   考虑使用高对比度的颜色组合来突出重要元素。
        *   可以探索渐变、阴影等效果来增加视觉深度。
    *   **布局：**
        *   使用基于网格的布局系统来组织页面元素。
        *   充分利用负空间（留白），创造视觉平衡和呼吸感。
        *   可以考虑使用卡片、分割线、图标等视觉元素来分隔和组织内容。
    *   **调性：**整体风格精致，营造一种高级感。
    *   **数据可视化：**
        *   设计一个或多个数据可视化元素，展示Naval思想的关键概念和它们之间的关系。
        *   可以考虑使用思想导图、概念关系图、时间线或主题聚类展示等方式。
        *   确保可视化设计既美观又有洞察性，帮助用户更直观地理解Naval思想体系的整体框
        *   使用Mermaid.js来实现交互式图表，允许用户探索不同概念之间的关联。
    **技术规范：**
    *   使用 HTML5、Font Awesome、Tailwind CSS 和必要的 JavaScript。
        *   Font Awesome: [https://cdn.bootcdn.net/ajax/libs/font-awesom
        *   Tailwind CSS: [https://g.alicdn.com/code/lib/tailwindcss/2.2
        *   非中文字体: [https://fonts.font.im/css?family=Noto+Serif+SC:wg
    this.onload=null"]
        *   `font-family: Tahoma,Arial,Roboto,"Droid Sans","Helvetica Ne
        *   Mermaid: [https://g.alicdn.com/code/lib/mermaid/11.5.0/merma
        *   Chart.js: [https://g.alicdn.com/code/lib/Chart.js/4.4.1/char
    *   代码结构清晰、语义化，包含适当的注释。
    *   实现完整的响应式，必须在所有设备上（手机、平板、桌面）完美展示。
    **额外加分项：**
    *   **微交互：** 添加微妙而有意义的微交互效果来提升用户体验（例如，按钮悬停效果、
    *   **补充信息：** 可以主动搜索并补充其他重要信息或模块（例如，关键概念的解释、相
    **输出要求：**
    *   提供一个完整、可运行的网页项目，其中包含所有必要的多个 HTML、CSS 和 JavaSc
    *   每一个文件内容尽量少，可以通过文件引用来避免重复，重复读/重复写会浪费token数
    *   确保代码符合 W3C 标准，没有错误或警告。
    *   在根目录下维护**项目大纲**文档，来记录你的设计思路、代码模块功能、文件内容概要
    <用户的问题>
计划9月15到9月24或25去西欧旅游，杭州出发亲子游，共9天。地点选择Amsterdam、Cologn
    </用户的问题>
    当前时间是2025年8月21日，我们开始吧！
```



6.4 引入Sub Agents实现业务逻辑

产品需求文档 -> 需求处理Agent -> UI设计Agent -> UI优化Agent -> 代码逻辑开发Agent -> 接口集成Agent -> 单元测试Agent

目前我们的合作团队已经实现了这样的CLI开发范式，待后续上线我们会公开具体细节，欢迎大家和我们交流合作!

更多最佳实践等你来发现!

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

968篇原创内容

公众号

阅读 6439